

Line Drill Deck

Sixty single lines with the seven answers and the note — cover the answers, say them, uncover

October 2026

How to drill. Cover everything below the code line. Say, out loud and in order: Kind · Source · Prim/Obj · Ticket? · Nothing? · Who else? · Kept? · then the note. Uncover and compare. Under a minute per line is the target. Assume the obvious context (`users: User[], orders: Order[], req` is an Express request, `cache` is a module-level `Map`, `getUser/save/load` are `async`).

A. Declarations and literals

1. `const count = 0;`

Kind	declaration (<code>const</code>)
Source	number literal
Prim/Obj	primitive
Ticket?	no call
Nothing?	no
Who else?	n/a (primitive)
Kept?	yes, in <code>count</code>
Note	<code>count</code> holds 0, a value; label fixed, but <code>count</code> is falsy

2. `const user = { id: 1, name: "Ann" };`

Kind	declaration
Source	object literal
Prim/Obj	object
Ticket?	no call
Nothing?	no
Who else?	new address; own
Kept?	yes
Note	<code>user</code> holds a new object; <code>user.name = "x"</code> is legal under <code>const</code>

3. `const ids: number[] = [];`

Kind	declaration with type
Source	array literal
Prim/Obj	object
Ticket?	no
Nothing?	no; empty is not nothing, and <code>[]</code> is truthy
Who else?	new; own
Kept?	yes
Note	<code>ids</code> holds a new empty array; <code>if (!ids)</code> never fires; use <code>ids.length === 0</code>

4. `let total;`

Kind	declaration, no value
Source	none
Prim/Obj	primitive (<code>undefined</code>)
Ticket?	no
Nothing?	yes, until assigned
Who else?	n/a
Kept?	n/a
Note	<code>total</code> holds <code>undefined</code> ; type is <code>any</code> unless annotated; <code>total += 1</code> gives <code>NaN</code>

5. `const { id, email } = user;`

Kind	declaration, destructuring
Source	property reads <code>user.id</code> , <code>user.email</code>
Prim/Obj	whatever those are (likely primitives)
Ticket?	no
Nothing?	yes if either key is optional or absent on <code>user</code>
Who else?	if a value is an object, shared with <code>user</code>
Kept?	yes, two labels
Note	<code>id</code> , <code>email</code> hold the field values; <code>email</code> may be <code>undefined</code> if <code>email?:</code>

6. `const [first, ...rest] = items;`

Kind	declaration, array destructuring
Source	<code>items[0]</code> and a new array of the rest
Prim/Obj	<code>first</code> is an element (shared); <code>rest</code> is a new array
Ticket?	no
Nothing?	<code>first</code> is <code>undefined</code> if <code>items</code> is empty
Who else?	<code>first</code> shared with <code>items[0]</code> ; <code>rest</code> own but its elements shared
Kept?	yes
Note	<code>first</code> holds <code>items[0]</code> or <code>undefined</code> ; <code>rest</code> holds a new array

7. `const opts = { ...DEFAULTS, ...input };`

Kind	declaration
Source	object literal with two spreads
Prim/Obj	object
Ticket?	no
Nothing?	no; individual keys may be
Who else?	new outer object; nested objects shared with <code>DEFAULTS/input</code> ; <code>input</code> keys win (correct order)
Kept?	yes
Note	<code>opts</code> holds a new object, input over defaults, shallow

8. `const opts = { ...input, ...DEFAULTS };`

Kind	declaration
Source	literal with two spreads
Prim/Obj	object
Ticket?	no
Nothing?	no
Who else?	new; shallow; <code>DEFAULTS</code> keys win, so user input is discarded
Kept?	yes
Note	<code>opts</code> holds defaults overriding input: BUG, order reversed

9. `const copy = original;`

Kind	declaration
Source	another label
Prim/Obj	object (assume)
Ticket?	no
Nothing?	if <code>original</code> can be
Who else?	ALIAS of <code>original</code> ; nothing copied
Kept?	yes
Note	<code>copy</code> holds the same object as <code>original</code> ; mutating <code>copy</code> mutates <code>original</code>

10. `const copy = { ...original };`

Kind	declaration
Source	spread literal
Prim/Obj	object
Ticket?	no
Nothing?	no
Who else?	new outer; <code>copy.inner</code> shared with <code>original.inner</code>
Kept?	yes
Note	<code>copy</code> holds a shallow copy; safe for top-level keys only

11. `const f = (x: number) => x * 2;`

Kind	declaration of a function
Source	arrow literal, expression body
Prim/Obj	object (function)
Ticket?	not async
Nothing?	no
Who else?	own
Kept?	yes
Note	<code>f</code> holds a function that returns <code>x * 2</code> ; <code>f(3)</code> is 6

12. `const f = (x: number) => { x * 2 };`

Kind	declaration of a function
Source	arrow literal, block body, no <code>return</code>
Prim/Obj	function
Ticket?	no
Nothing?	<code>f(3)</code> returns <code>undefined</code>
Who else?	own
Kept?	yes
Note	<code>f</code> holds a function that returns <code>undefined</code> : BUG, braces without return

13. `const f = async (x: number) => x * 2;`

Kind	declaration
Source	async arrow
Prim/Obj	function
Ticket?	calling it returns <code>Promise<number></code>
Nothing?	no
Who else?	own
Kept?	yes
Note	<code>f(3)</code> is a ticket for 6; <code>await f(3)</code> is 6

14. `interface User { id: number; email?: string }`

Kind	type-only
Source	n/a
Prim/Obj	n/a
Ticket?	n/a
Nothing?	the claim: <code>email</code> may be absent
Who else?	n/a
Kept?	n/a
Note	runs nothing; every <code>user.email</code> downstream is <code>string undefined</code>

15. `const key = req.headers["x-api-key"] as string;`

Kind	declaration with a cast
Source	property read from a request
Prim/Obj	primitive (or <code>string[]</code> , or <code>undefined</code>)
Ticket?	no
Nothing?	YES; header may be absent; the <code>as</code> hides it
Who else?	n/a
Kept?	yes
Note	<code>key</code> holds a string OR <code>undefined</code> OR an array; the cast is a lie; next use must check

B. Assignments and mutations

16. `total = total + order.amount;`

Kind	reassignment
Source	arithmetic
Prim/Obj	primitive number
Ticket?	no
Nothing?	<code>NaN</code> if <code>order.amount</code> is <code>undefined</code> or a non-numeric string
Who else?	n/a
Kept?	yes
Note	<code>total</code> holds the sum; poisoned to <code>NaN</code> by one bad amount; floats drift for money

17. `user.name = name.trim();`

Kind	property set (mutation)
Source	string method (returns new)
Prim/Obj	string stored into an object
Ticket?	no
Nothing?	<code>name</code> must not be <code>undefined</code> or <code>.trim</code> crashes
Who else?	mutates <code>user</code> ; every alias of <code>user</code> sees the new name
Kept?	yes, the mutation
Note	<code>user.name</code> is now the trimmed string; <code>user</code> shared with whoever passed it

18. `orders.sort((a, b) => a.total - b.total);`

Kind	expression statement, mutating call
Source	<code>sort</code> with numeric comparator
Prim/Obj	object (array)
Ticket?	no
Nothing?	no
Who else?	MUTATES <code>orders</code> in place; if <code>orders</code> is a parameter, the caller's array is reordered
Kept?	the mutation is the effect
Note	<code>orders</code> is now sorted ascending by total; finding if the input must not change

19. `const sorted = orders.sort();`

Kind	declaration
Source	<code>sort</code> with no comparator
Prim/Obj	array
Ticket?	no
Nothing?	no
Who else?	<code>sorted === orders</code> ; not a copy; and objects sort by <code>"[object Object]"</code>
Kept?	kept, but misleading
Note	<code>sorted</code> holds the same array as <code>orders</code> , both reordered meaninglessly: BUG twice

20. `const sorted = [...orders].sort((a, b) => a.total - b.total);`

Kind	declaration
Source	spread copy, then sort
Prim/Obj	array
Ticket?	no
Nothing?	no
Who else?	new array; <code>orders</code> untouched; elements still shared
Kept?	yes
Note	<code>sorted</code> holds a new sorted array; correct pattern

21. `arr.length = 0;`

Kind	assignment to a property
Source	literal 0
Prim/Obj	mutates an array
Ticket?	no
Nothing?	no
Who else?	every alias of <code>arr</code> is now empty
Kept?	the mutation
Note	<code>arr</code> emptied in place

22. `delete cache[key];`

Kind	expression statement (mutation)
Source	<code>delete</code> operator
Prim/Obj	mutates an object
Ticket?	no
Nothing?	if <code>key</code> is <code>undefined</code> , deletes the key <code>"undefined"</code>
Who else?	shared module state
Kept?	the mutation
Note	key removed; if <code>cache</code> is a <code>Map</code> , this does nothing and <code>cache.delete(key)</code> was meant

23. `count ??= 0;`

Kind	logical assignment
Source	<code>??=</code>
Prim/Obj	primitive
Ticket?	no
Nothing?	after this line, no
Who else?	n/a
Kept?	yes
Note	<code>count</code> is 0 only if it was <code>null/undefined</code> ; an existing 0 stays 0 (correct, unlike <code> =</code>)

24. `if (order.status = "paid") paid.push(order);`

Kind	control with an assignment inside the condition
Source	assignment expression; value <code>"paid"</code>
Prim/Obj	primitive
Ticket?	no
Nothing?	<code>"paid"</code> is truthy; branch always runs
Who else?	mutates <code>order</code> , an element of the caller's array
Kept?	the condition is used, but always true
Note	every order's status becomes <code>"paid"</code> and every order is pushed: BUG, <code>=</code> for <code>===</code>

C. Reads and lookups

25. `const user = users.find((u) => u.id === id);`

Kind	declaration
Source	<code>find</code>
Prim/Obj	object or <code>undefined</code>
Ticket?	no
Nothing?	YES on no match; also if <code>id</code> is a string and <code>u.id</code> a number, always undefined
Who else?	shared with the element inside <code>users</code>
Kept?	yes
Note	<code>user</code> holds an element of <code>users</code> or <code>undefined</code> ; next <code>.</code> must check

26. `const name = user!.name;`

Kind	declaration with non-null assertion
Source	property read through <code>!</code>
Prim/Obj	primitive
Ticket?	no
Nothing?	the <code>!</code> claims no; nothing checks; crashes if <code>user</code> is undefined
Who else?	n/a
Kept?	yes
Note	<code>name</code> holds the string if <code>user</code> exists; the <code>!</code> is an unverified claim

27. `const name = user?.profile?.name ?? "anon";`

Kind	declaration
Source	optional chain with nullish fallback
Prim/Obj	primitive
Ticket?	no
Nothing?	no; <code>"anon"</code> fills every hole
Who else?	n/a
Kept?	yes
Note	<code>name</code> holds a string, always; correct defensive read

28. `const list = index.get(tag);`

Kind	declaration
Source	<code>Map.get</code>
Prim/Obj	object (array) or <code>undefined</code>
Ticket?	no
Nothing?	YES when the tag is new
Who else?	shared with the value inside the map
Kept?	yes
Note	<code>list</code> holds the map's array or <code>undefined</code> ; <code>list.push</code> crashes on a new tag

29. `index.get(tag).push(doc);`

Kind	expression statement
Source	<code>get</code> then <code>push</code>
Prim/Obj	array inside the map
Ticket?	no
Nothing?	<code>get</code> is <code>undefined</code> for a new tag; <code>.push</code> throws
Who else?	mutates the map's array
Kept?	the mutation

Note	crashes on first new tag; BUG; use <code>?? []</code> and <code>set</code>
------	--

30. `const last = arr[arr.length - 1];`

Kind	declaration
Source	index read
Prim/Obj	element (shared) or <code>undefined</code>
Ticket?	no
Nothing?	yes if <code>arr</code> is empty (<code>arr[-1]</code>)
Who else?	shared with the element
Kept?	yes
Note	<code>last</code> holds the last element or <code>undefined</code>

31. `const size = Number(req.query.size);`

Kind	declaration
Source	conversion call
Prim/Obj	primitive number
Ticket?	no
Nothing?	never <code>undefined</code> ; <code>NaN</code> if absent or non-numeric; <code>0</code> if <code>"0"</code>
Who else?	n/a
Kept?	yes
Note	<code>size</code> holds a number, possibly <code>NaN</code> ; a range check alone will not reject <code>NaN</code>

32. `const size = Number(req.query.size) || 20;`

Kind	declaration
Source	conversion with <code> </code> fallback
Prim/Obj	number
Ticket?	no
Nothing?	no; but <code>0</code> and <code>NaN</code> both become 20
Who else?	n/a
Kept?	yes
Note	<code>size</code> holds 20 for <code>?size=0</code> (wrong) and for <code>?size=abc</code> (silently); negative passes

33. `const orgId = req.query.orgId ?? claims.orgId;`

Kind	declaration
Source	<code>??</code> between request input and token claim
Prim/Obj	primitive
Ticket?	no
Nothing?	no if claims has it
Who else?	n/a
Kept?	yes
Note	<code>orgId</code> holds whatever the CALLER sent, overriding the token: AUTH BYPASS; use <code>claims.orgId</code> only

34. `for (const i in items) total += items[i + 1];`

Kind	control, <code>for...in</code> on an array
Source	index keys as strings
Prim/Obj	<code>i</code> is a string
Ticket?	no
Nothing?	<code>items["01"]</code> is <code>undefined</code>
Who else?	n/a
Kept?	<code>total</code> becomes <code>NaN</code>

Note	<code>i</code> holds <code>"0"</code> , <code>"1"</code> ; <code>i + 1</code> is <code>"01"</code> : BUG, use <code>for...of</code> or a counting loop
------	--

35. `for (const [k, v] of Object.entries(counts)) {`

Kind	control, <code>for...of</code> with destructuring
Source	<code>Object.entries</code> (new array of pairs)
Prim/Obj	<code>k</code> string, <code>v</code> whatever the values are
Ticket?	no
Nothing?	no
Who else?	<code>v</code> shared with <code>counts[k]</code> if an object
Kept?	n/a
Note	<code>k</code> , <code>v</code> hold each key and value; the Python <code>d.items()</code> form

D. Calls, sync

36. `const paid = orders.filter((o) => o.status === "paid");`

Kind	declaration
Source	<code>filter</code>
Prim/Obj	new array
Ticket?	no
Nothing?	no; possibly empty
Who else?	new array; elements shared with <code>orders</code>
Kept?	yes
Note	<code>paid</code> holds a new array of the same order objects

37. `orders.filter((o) => o.status === "paid");`

Kind	expression statement
Source	<code>filter</code>
Prim/Obj	new array
Ticket?	no
Nothing?	no
Who else?	nothing mutated
Kept?	NO; dropped
Note	dead line; nothing happened

38. `const amounts = orders.map((o) => { o.amount });`

Kind	declaration
Source	<code>map</code> with block body, no <code>return</code>
Prim/Obj	array of <code>undefined</code>
Ticket?	no
Nothing?	every element is <code>undefined</code>
Who else?	new array
Kept?	yes, but useless
Note	<code>amounts</code> holds <code>[undefined, ...]</code> : BUG, braces without return

39. `const total = amounts.reduce((a, b) => a + b);`

Kind	declaration
Source	<code>reduce</code> without initial value
Prim/Obj	number
Ticket?	no
Nothing?	THROWS on empty <code>amounts</code>

Who else?	n/a
Kept?	yes
Note	<code>total</code> holds the sum, or a <code>TypeError</code> on <code>[]</code> ; add <code>, 0</code>

40. `const n = arr.push(x);`

Kind	declaration
Source	<code>push</code> (mutating; returns length)
Prim/Obj	number
Ticket?	no
Nothing?	no
Who else?	<code>arr</code> mutated
Kept?	yes, but it is the length, not the array
Note	<code>n</code> holds the new length; <code>arr</code> has <code>x</code> appended

41. `name.trim();`

Kind	expression statement
Source	string method
Prim/Obj	new string
Ticket?	no
Nothing?	n/a
Who else?	nothing mutated (strings immutable)
Kept?	NO
Note	dead line; <code>name</code> unchanged

42. `const data = JSON.parse(body);`

Kind	declaration
Source	<code>JSON.parse</code> (sync)
Prim/Obj	anything; typed <code>any</code>
Ticket?	no
Nothing?	<code>null</code> if body is <code>"null"</code> ; throws if malformed
Who else?	new
Kept?	yes
Note	<code>data</code> holds an unvalidated value of unknown shape; needs <code>try</code> and a runtime check

43. `const data = JSON.parse(body) as Config;`

Kind	declaration with cast
Source	<code>JSON.parse</code>
Prim/Obj	any
Ticket?	no
Nothing?	same as 42; the cast changes nothing
Who else?	new
Kept?	yes
Note	<code>data</code> holds whatever the client sent, labeled <code>Config</code> with no check: TYPE HOLE

44. `const ok = users.includes(user);`

Kind	declaration
Source	<code>includes</code> on objects
Prim/Obj	boolean
Ticket?	no
Nothing?	no

Who else?	n/a
Kept?	yes
Note	ok is <code>true</code> only if that exact object (same address) is in <code>users</code> ; a fresh <code>{ id: 1 }</code> is never found

45. `const d = new Date(order.createdAt);`

Kind	declaration
Source	constructor
Prim/Obj	object (Date), mutable
Ticket?	no
Nothing?	<code>Invalid Date</code> if the input is garbage; <code>d.getTime()</code> is <code>NaN</code>
Who else?	new
Kept?	yes
Note	<code>d</code> holds a Date; compare with <code>.getTime()</code> , never <code>===</code>

E. Calls, async

46. `const user = getUser(id);`

Kind	declaration
Source	async call
Prim/Obj	object: a Promise
Ticket?	TICKET; no <code>await</code>
Nothing?	<code>user.name</code> is <code>undefined</code>
Who else?	new Promise
Kept?	kept, as a ticket
Note	<code>user</code> holds <code>Promise<User></code> , not awaited: BUG

47. `const user = await getUser(id);`

Kind	declaration
Source	awaited async call
Prim/Obj	object (User)
Ticket?	unwrapped
Nothing?	only if <code>getUser</code> can resolve to <code>null/undefined</code>
Who else?	depends on <code>getUser</code> ; if it returns a cached object, shared
Kept?	yes
Note	<code>user</code> holds a User; this line is a yield point

48. `save(data);`

Kind	expression statement
Source	async call
Prim/Obj	Promise
Ticket?	ticket, dropped
Nothing?	n/a
Who else?	n/a
Kept?	NO
Note	fire-and-forget; rejection unhandled; function continues before save finishes

49. `const data = res.json();`

Kind	declaration
Source	<code>res.json()</code>
Prim/Obj	Promise

Ticket?	ticket, not awaited
Nothing?	<code>data.user</code> is <code>undefined</code>
Who else?	new
Kept?	as a ticket
Note	<code>data</code> holds <code>Promise<any></code> : BUG, missing <code>await</code>

50. `const res = await fetch(url); const user = await res.json();`

Kind	two declarations
Source	<code>fetch</code> , then <code>res.json()</code>
Prim/Obj	Response, then any
Ticket?	both awaited
Nothing?	<code>res.ok</code> unchecked: a 404 body is parsed as <code>user</code>
Who else?	new
Kept?	yes
Note	<code>user</code> holds the parsed body of ANY status; needs <code>if (!res.ok) throw</code> between them

51. `const users = ids.map((id) => getUser(id));`

Kind	declaration
Source	<code>map</code> with an async callback
Prim/Obj	array of Promises
Ticket?	an ARRAY of tickets
Nothing?	<code>users[0].name</code> is <code>undefined</code>
Who else?	new array
Kept?	yes, but as tickets
Note	<code>users</code> holds <code>Promise<User>[]</code> ; needs <code>await Promise.all(...)</code>

52. `const users = await Promise.all(ids.map((id) => getUser(id)));`

Kind	declaration
Source	<code>Promise.all</code> over mapped Promises
Prim/Obj	array of Users
Ticket?	all unwrapped, in input order
Nothing?	rejects entirely on the first failure
Who else?	new array
Kept?	yes
Note	<code>users</code> holds <code>User[]</code> ; parallel; correct

53. `ids.forEach(async (id) => { await save(id); });`

Kind	expression statement
Source	<code>forEach</code> with async callback
Prim/Obj	<code>forEach</code> returns <code>undefined</code>
Ticket?	each callback's Promise is dropped by <code>forEach</code>
Nothing?	n/a
Who else?	n/a
Kept?	NO; nothing waits
Note	the saves run detached; the function continues immediately: BUG

54. `for (const id of ids) await save(id);`

Kind	control
Source	awaited call per iteration
Prim/Obj	n/a

Ticket?	each awaited; sequential
Nothing?	n/a
Who else?	n/a
Kept?	the side effect
Note	saves run one at a time; correct if order matters, slow if not

55. `try { save(data); } catch (e) { log(e); }`

Kind	control, <code>try</code>
Source	async call inside, not awaited
Prim/Obj	Promise dropped
Ticket?	not awaited inside the <code>try</code>
Nothing?	n/a
Who else?	n/a
Kept?	NO
Note	the <code>try</code> ends before the Promise settles; <code>catch</code> never runs: BUG, <code>await save(data)</code>

56. `return fetch(url);` (inside a `try` in a non-async function)

Kind	control, <code>return</code>
Source	<code>fetch</code> (Promise)
Prim/Obj	Promise
Ticket?	handed to the caller un-awaited
Nothing?	n/a
Who else?	n/a
Kept?	returned
Note	the Promise leaves the <code>try</code> pending; this function's <code>catch</code> is dead; caller must handle

57. `return await fetch(url);` (inside a `try` in an async function)

Kind	control, <code>return</code> with <code>await</code>
Source	<code>fetch</code> , awaited
Prim/Obj	Response
Ticket?	unwrapped here
Nothing?	n/a
Who else?	n/a
Kept?	returned
Note	a rejection is thrown inside this <code>try</code> ; <code>catch</code> runs; the one place <code>return await</code> matters

58. `const n = counts.get(k) ?? 0; await db.save(k); counts.set(k, n + 1);`

Kind	three statements: read, await, write
Source	<code>Map.get</code> , async call, <code>Map.set</code>
Prim/Obj	number; shared Map
Ticket?	<code>save</code> awaited
Nothing?	<code>get</code> miss handled by <code>?? 0</code>
Who else?	<code>counts</code> is shared; another request runs at the <code>await</code>
Kept?	the write
Note	read-await-write on shared state: RACE; concurrent requests lose increments

59. `if (!cache.has(k)) cache.set(k, await load(k));`

Kind	control with an awaited call inside
Source	<code>has</code> , then <code>load</code> awaited, then <code>set</code>
Prim/Obj	shared Map

Ticket?	<code>load</code> awaited
Nothing?	n/a
Who else?	<code>cache</code> shared; every concurrent miss passes <code>has</code> before any <code>set</code>
Kept?	the write
Note	check-then-set across an <code>await</code> : all concurrent misses call <code>load</code> ; store the Promise instead

60. `setInterval(sweep, 60_000);`

Kind	expression statement
Source	timer call
Prim/Obj	returns a timer id (number/object), dropped
Ticket?	not a Promise
Nothing?	n/a
Who else?	the timer holds <code>sweep</code> and its closure forever
Kept?	id dropped, so it can never be cleared
Note	runs every minute, keeps the process alive, cannot be cancelled: needs <code>.unref()</code> or a stored id

Answer key in one table

#	Verdict	Which Q caught it
1–3, 5–7, 10, 11, 13, 14, 20, 23, 27, 35, 36, 40, 44, 45, 47, 52, 54, 57	fine (with the stated caveat)	
4	<code>undefined</code> until assigned	Q5
8	spread order reversed	Q6
9	alias, not copy	Q6
12	braces without return	Q5/Q7
15	cast hides <code>undefined</code>	Q5
16	<code>NaN</code> poisoning; float money	Q5
17	mutates shared <code>user</code>	Q6
18	mutates caller's array	Q6
19	<code>sort()</code> not a copy, string order	Q6/Q7
21, 22	mutation reaches all aliases; <code>delete</code> on a Map does nothing	Q6
24	<code>=</code> in condition	Q2/Q5
25, 28, 30	can be <code>undefined</code>	Q5
26	<code>!</code> unverified	Q5
29	<code>get().push</code> crash	Q5
31, 32	<code>NaN</code> ; <code> </code> eats 0	Q5
33	auth bypass via <code>??</code>	Q2 (source is the caller)
34	<code>for...in</code> string index	Q2/Q5
37, 41	result dropped	Q7
38	braces without return	Q5
39	<code>reduce</code> on empty throws	Q5
42, 43	unvalidated parse; cast	Q5
46, 49, 51	ticket not awaited	Q4
48, 53	fire-and-forget	Q4/Q7
50	<code>res.ok</code> unchecked	Q5
55, 56	<code>try</code> without <code>await</code> inside	Q4
58, 59	race across <code>await</code>	Q6
60	timer never cleared	Q6/Q7